Developing a Computer Simulation for Estimating Lowest-Energy VSEPR Geometries

11-MTH-005

Abstract

Valence shell electron pair repulsion theory (VSEPR theory) predicts the geometry of covalent-bonded molecules given the amount of electron pairs located around the central atom's valence (outer) shell. Electron pairs will repel one another according to Coulomb's law and reach a minimal-energy state, thereby resulting in the most stable configuration. There are two types of electron pairs: bonded pairs (two atoms provide the two electrons) and lone pairs (only the central atom provides the two electrons), which often exert more energy than bonded pairs. The goal of this research was to develop and use an optimization program that could produce the minimal energy value and geometries for arbitrary N electron pairs. Using numerically conjectured energy values from an online database, the algorithm reproduced the same approximate values with minimal error. Then, the program was applied to cases incorporating lone-pair electrons, where no solid data exists, and the lowest values from resulting trials were compiled in a dataset. Although the findings do not formally prove or provide an exact solution to this optimization problem, they offer a computationally-derived approximation that may serve as a foundation for further theoretical and experimental investigation.

1 Methodology

1.1 Part 1: Mathematical Analysis

From Coulomb's law, the energy of a system containing N electron pairs can be expressed as

$$E(i,j) = \frac{e_i e_j}{4\pi\epsilon_0 r_{ij}}, \tag{1}$$

where e_i and e_j are the electrostatic charges of the electron pairs, ϵ_0 is the permittivity of free space, and r_{ij} is the distance between the two electron pairs.

In the case where all electron pairs are bonded, e becomes a constant value.

Via the Thomson problem, (1) simplifies further by eliminating the constant values:

$$E(i,j) = \frac{1}{r_{ij}}. \tag{2}$$

Rewriting the function to account for the total energy in the system, we have

$$\sum E(i,j) = \sum_{i,j} \frac{1}{r_{ij}}. \tag{3}$$

If r_{ij} were written in terms of rectangular coordinates, (3) would be a function expressed in terms of $3N$ variables, N being the number of electron pairs in the system. However, converting the expression into spherical coordinates reduces the number of variables to $2N$ (as all points lie on a sphere's surface, making coordinate ρ a constant value for all such points). Converting r_{ij} into spherical coordinates [with the convention θ being the azimuthal angle and ϕ being the polar angle], we have

$$r_{ij} = \sqrt{2 - 2\left(\sin\phi_i \sin\phi_j \cos(\theta_i - \theta_j) + \cos\phi_i \cos\phi_j\right)}. \tag{4}$$

Now, we have the function $\sum E(i,j)$ expressed in terms of $2N$ variables. The algorithm utilizes this function's gradient to perform gradient descent. Solving for both gradients, in terms of a single coordinate pair (θ_i, ϕ_i), we have

$$\frac{\partial E}{\partial \theta_i} = \sum_{j=0, j\neq i}^{N} \frac{\sin\phi_i \sin\phi_j \sin(\theta_i - \theta_j)}{r_{ij}^3},$$

$$\frac{\partial E}{\partial \phi_i} = \sum_{j=0, j\neq i}^{N} \frac{\sin\phi_i \cos\phi_j - \cos\phi_i \sin\phi_j \cos(\theta_i - \theta_j)}{r_{ij}^3}.$$

1.2 Part 2; Algorithms and Comparative Analysis

The algorithm was implemented in Python, using modules such as NumPy and Open3D to perform fast vectorized operations and visualize the points on a sphere respectively. Using a slight variation of gradient descent, the actual optimization was divided into three steps:

Step A: This step involves an algorithm known as *simulated annealing*, named after the parent process of annealing in metallurgy and materials science. Before this step takes place, uniformly random points are generated and normalized, and those points are the input to this looping step. Two more variables are defined in this loop: T for temperature and D for decay. T and D are predetermined before the algorithm starts, and T scales with higher N to increase the amount of loops the steps will need for higher, more complex Ns

First, the algorithm calculates numerical gradient values for each point, then subtracts these values from the original points. Then, $\lfloor \frac{N}{5} \rfloor$ points are randomly selected, and random noise is applied onto those points. This is to encourage the algorithm to escape local minima and attempt to reach global minima, as (3) is a non-convex function. Then, the algorithm calculates the energy value of the new configuration. If the energy value is lower than the previous value, the new configuration is accepted. However, if the new energy value is higher than the previous value, the new configuration is accepted with a probability by a probability acceptance function (PAF):

$$\mathrm{PAF}(E_{\mathrm{new}}, E_{\mathrm{old}}, T) = \exp\left(\frac{E_{\mathrm{old}} - E_{\mathrm{new}}}{T}\right).$$

Then, T gets decayed by a factor of D after each iteration. The temperature T acts like a controlling parameter here: it will allow the algorithm to take on more high-energy configurations at the beginning of the optimization, but it becomes more restrictive later on as T decays. At the end of the program, the configuration that resulted in the lowest energy is used in the next step.

Step O: This step is a more traditional gradient descent step, where the algorithm now simply calculates the gradient for all points and subtracts them from the points. The new configuration is only accepted if the energy is lower than the previous. The factor by which the gradient is multiplied evolves; it increases after a large majority of configuration changes (i.e., how many times the new energy value is lower than the old energy value) and decreases after a large majority of rejections. This is to fine-tune the optimal energy value, as the program slowly converges to the minimum. This step runs for a set amount of steps, and the program worked well if it ran for the same amount of iterations as **Step A**.

Step R: This step applies the same traditional gradient descent as **Step O**, but the gradient is applied to a single point only per iteration. The point is uniformly chosen at random, and the new configuration is only accepted if the energy value is lower than the previous. This is to provide as a fine-tuning step, finding better configurations that the previous steps may have missed due to the nature of the optimization. Like the previous step, this loops for a set amount of iterations and worked well using the same iterations as the previous steps.

To test the algorithm's efficiency, the algorithm was applied to three ranges: a low ($N \in [2, 50]$), medium ($N \in [90, 100]$), and high ($N \in [140, 150]$) range, and each N value was run through the algorithm ten times. Fewer values were tested for higher N due to the algorithm's increasing computational cost. The computed energies were validated against accepted energy values from the Cambridge Cluster Database (Wales & Ulker, 2006). Lower energy

values were compared using values from the Neil Sloane database (Hardin, Sloane, & Conway, 1994).

1.3 Part 3: Lone-Pair Application

First, we need to re-reference (1):

$$E(i,j) = \frac{e_i e_j}{4\pi\epsilon_0 r_{ij}}. \tag{5}$$

When lone pairs are involved, we can no longer assume that e is a constant value. Instead, we will have to assign a value to e for both bonded and lone pairs. Let e for a bonded pair to be 1. Using an iterative decimal expansion search, two approximate values for e for a lone pair were found: knowing that water's bond angle is approximately 104.5 degrees, the value of e for a lone pair was found to be approximately 1.244217559: in other words, using the structure of water ($N_b = 2, N_l = 2$), $e = 1.244217559$ will produce a near-exact 104.5-degree angle. Then, using the structure of ammonia ($N_b = 3, N_l = 1$), e for a lone pair was found to be 1.220931786. The algorithm used the mean, $e = 1.23257467$, as the value for e for a lone pair, and the program was applied to lone-pair configurations.
To analyze for any relationships between the number of lone pairs and the energy values (compared to the bonded-pair-only configurations), values were gathered for configurations with bonded pairs ranging from $[2, 50]$ and lone pairs ranging from $[2, 5]$, with ten trials for each configuration. The lowest energy value was then compiled into a dataset, and linear regression was performed on the data to analyze for potential relationships.

2 Results

The algorithm was able to reproduce the same energy values as the Cambridge Cluster Database with minimal error (values differed by less than 0.1 for all tested N values). Due to local minima, the algorithm sometimes failed to converge to the global minimum for some N values. More iterations of the program should be able to counteract this issue, but due to the complicated graph of the energy function for higher N values, the algorithm could be further optimized.
For lone pairs, Table 3 below shows the lowest energy values for each configuration, generated by the algorithm. Linear regression was performed on the data, and the results are as follows:

Table 1: Linear regression for energy difference vs total electron pairs (lone pair counts 2–5).

lone pairs	slope	intercept	r	p-value	stderr
2	0.408 010 29	−1.028 747 77	0.9997	5.8669×10^{-81}	0.001 404 79
3	0.614 785 26	−1.556 218 78	0.9998	1.1603×10^{-83}	0.001 864 04
4	0.822 987 31	−2.081 359 91	0.9998	6.4415×10^{-86}	0.002 244 26
5	1.032 225 28	−2.594 827 75	0.9998	6.8041×10^{-88}	0.002 565 14

Table 2: Regression of slope and intercept coefficients vs lone pairs, n.

Parameter	Slope	Intercept	r	p-value	stderr
a_n	0.208 084 70	−0.008 794 42	0.999 996 48	3.5202×10^{-6}	0.000 390 42
b_n	−0.522 338 11	0.012 894 82	0.999 980 43	1.9567×10^{-5}	0.002 310 57

The results show a very strong linear correlation between difference in energy values (compared to the bonded-pair-only configuration) and the number of bonded pairs, with all r-values above 0.999 and all p-values significantly below $\alpha = 0.05$. The slope and y-intercept of the linear regression also increases with the number of lone pairs, and a linear regression on the slopes themselves also shows a strong linear correlation. These results suggest that lone pairs influence the energy of the system in a linear manner, and the influence of lone pairs increases as the number of lone pairs increases. This leads to a general formula: for M bonded electron pairs and N lone electron pairs, the energy of the system can be approximated as follows:

$$\begin{aligned} E(m,n) = E_0(m) &+ (0.20808470 \cdot n - 0.00879442) \cdot m \\ &+ (-0.52233811 \cdot n + 0.01289482) \end{aligned} \tag{6}$$

The intercept values of 0.00079442 and 0.01209482 could be ignored as they are relatively close to 0:

$$E(m,n) = E_0(m) + 0.20808470 \cdot mn + -0.52233811 \cdot n \tag{7}$$

This is a surprising discovery: the only values that need to be calculated are bonded pair-only energy values, and the exact value for e for a lone pair electron (due to this paper approximating this value based off of two estimates). Because of the inaccuracy of the lone pair e value, the results of this paper should be taken with a grain of salt, but the strong linear correlations suggest that there is a linear relationship that may be further investigated in the future.

Table 3: Bonded-n vs energy for lone-pairs of 2, 3, 4, 5

2 lone pairs		3 lone pairs		4 lone pairs		5 lone pairs	
bonded	energy	bonded	energy	bonded	energy	bonded	energy
0	0.75962016	0	2.63140142	0	5.58204536	0	9.83661236
1	2.29812046	1	5.05161335	1	9.07856893	1	14.20884042
2	4.55715045	2	8.36077286	2	13.28621255	2	19.59061979
3	7.69486164	3	12.40223069	3	18.47092483	3	25.74130500
4	11.55869516	4	17.39356366	4	24.43751769	4	32.74930780
5	16.35714884	5	23.18877664	5	31.26919045	5	40.66250806
6	21.97634835	6	29.82796391	6	38.98807882	6	49.49415038
7	28.42969132	7	37.35710669	7	47.62526314	7	59.04500785
8	35.76915749	8	45.79891984	8	56.98466155	8	69.68221488
9	44.02061199	9	54.96457894	9	67.41966786	9	81.11853170
10	52.98970669	10	65.20485859	10	78.65621292	10	93.46132860
11	63.03300156	11	76.23843241	11	90.79921071	11	106.67444749
12	73.89409201	12	88.18166220	12	103.83858190	12	120.83340963
13	85.63444618	13	101.04764029	13	117.78900003	13	135.89523707
14	98.27365412	14	114.79107270	14	132.63988745	14	151.87645171
15	111.83088656	15	129.42927622	15	148.42760299	15	168.70742587
16	126.27237678	16	145.02300347	16	165.04947573	16	186.47195830
17	141.66452737	17	161.44139587	17	182.61578910	17	205.15901327
18	157.87486355	18	178.79767556	18	201.09483546	18	224.80444585
19	175.03206911	19	197.07122697	19	220.53260669	19	245.27609740
20	193.09895473	20	216.31381347	20	240.79310742	20	266.74475871
21	212.13990877	21	236.36487813	21	262.06646335	21	289.08986643
22	231.97996676	22	257.42812920	22	284.20880243	22	312.31951831
23	252.84057669	23	279.36221275	23	307.22279056	23	336.53261985
24	274.57527002	24	302.17321785	24	331.22712553	24	361.69672690
25	297.16955175	25	325.97356631	25	356.18417397	25	387.71900224
26	320.76570320	26	350.72883063	26	381.99874316	26	414.69262495
27	345.30567647	27	376.33502936	27	408.76759026	27	442.49681003
28	370.70773206	28	402.87722408	28	436.34921029	28	471.42775277
29	397.04964289	29	430.25498305	29	465.08562238	29	501.17197768
30	424.21093117	30	458.78420290	30	494.61881721	30	531.89672045
31	452.52962793	31	488.12067876	31	525.12240627	31	563.50738941
32	481.66504772	32	518.40409088	32	556.51784044	32	596.04874911
33	511.73296223	33	549.59649571	33	588.86366255	33	629.54350522
34	542.71188332	34	581.73771347	34	622.12645228	34	663.94249750
35	574.63998367	35	614.77220086	35	656.33546886	35	699.29514079
36	607.48736686	36	648.77593319	36	691.47406173	36	735.58991382
37	641.26515651	37	683.70379778	37	727.56812825	37	772.81239958
38	675.97926053	38	719.57671046	38	764.56991925	38	810.97811116
39	711.63893567	39	756.37131594	39	802.52501354	39	850.03235035
40	748.22627018	40	794.11824304	40	841.35815735	40	890.10123776
41	785.75634701	41	832.73742932	41	881.23088387	41	931.14935355
42	824.17602888	42	872.39039569	42	922.05210778	42	973.08312485
43	863.60737780	43	913.00056708	43	963.78555584	43	1015.84281257
44	903.99797010	44	954.51785922	44	1006.31257786	44	1059.72381643
45	945.30398390	45	996.84260760	45	1049.99345203	45	1104.42056388
46	987.41722973	46	1040.31339900	46	1094.47535479	46	1150.11293873
47	1030.67642875	47	1084.57902755	47	1139.95441921	47	1196.79129313
48	1074.73117962	48	1129.84428106	48	1186.42434394	48	1244.36827995
49	1119.78650448	49	1176.08941135	49	1233.77097198	49	1292.87452398
50	1165.81944902	50	1223.22698877	50	1282.08195342	50	1342.37761259

References

[1] Wales, D. J., & Ulker, S. (2006). Cambridge Cluster Database. https://www-wales.ch.cam.ac.uk/CCD.html

[2] Hardin, R. H., Sloane, N. J. A., & Conway, J. H. (1994). Tables of energy minima for point charges on the sphere (Neil Sloane database). https://neilsloane.com/packings/